



Granular Acceptance Criteria

Week 3 • Day 2 • Requirements Engineering & Mini-Capstone

TPM Academy



Day 2 Objectives

- Write acceptance criteria (AC) in **Given/When/Then** form — testable, falsifiable, scoped
- Detect and rewrite the **5 AC failure modes**
- Cover **happy path / sad path / weird path** — the discipline that distinguishes TPM AC from PM AC
- Conduct an **AC cross-review** with another quad
- Add a **review-resolution note** documenting what was adopted, deferred, or rejected



Today's Shape

Clock	Block	Outcome
09:00 – 09:15	Opening	Restate non-AI rule; recap Day 1 output
09:15 – 10:30	Block 1 + Activity 1	AC Triage
10:45 – 12:00	Block 2 + Activity 2	Happy-path AC
13:00 – 14:15	Block 3 + Activity 3	Sad & weird-path AC
14:30 – 16:00	Block 4 + Activity 4 + Wrap	AC Cross-Review

Block 1 — What an AC Is

And the 5 ways AC most often fail



The Given/When/Then Form

```
Given <a precondition / starting state>  
When <an action or event happens>  
Then <an observable, falsifiable result>
```

Is	Is not
Testable (you can write a test that passes/fails)	"The system should be intuitive"
Specific to one behavior	"The system handles errors well"
Implementation-agnostic	"Use Redis to cache the response"
Falsifiable (you can prove it wrong)	"Performance should be good"



The Five AC Failure Modes

Failure	Example	Fix
1. Vague	"Then it loads quickly"	Replace with measurable target
2. Untestable	"Then user feels confident"	Anchor to system-visible state
3. Restating goal	"Then dispatchers reconcile faster"	Pin to in-system event, not metric
4. AND-soup	"Then X and Y and Z and W"	Split into multiple ACs
5. Implementation-prescriptive	"Redis cache hit returns..."	Behavior, not implementation

Activity 1 — AC Triage

Quads • 45 minutes

You receive 12 AC examples. For each, identify which failure mode(s) are present (or "clean"). Rewrite the failed ones.

- 20 min: triage all 12
- 10 min: rewrite the failed ones
- 5 min: identify the most common failure mode in your pack



20 triage • 10 rewrite • 5 reflect

Block 2 — Happy-Path AC

Easiest first; get them out of the way

Happy / Sad / Weird Path Coverage

Path	Question	Typical AC count
Happy	What does success look like end-to-end?	3–5
Sad	What happens when input is wrong, user lacks permission, etc.?	2–4
Weird	What about network drops, race conditions, edge cases the user wouldn't think of?	2–4

The TPM differentiator: the weird-path coverage. PMs who don't think technically miss it. Engineers who don't think about users miss it differently.

Worked Happy-Path Example

```
Given a dispatcher viewing the end-of-shift summary
When they tap "Reconcile all"
Then the reconcile modal opens within 1 second
and shows all open tickets pre-selected
and the header shows the count of tickets selected
```

Note: the multiple "and"s here describe one observable state, not separate behaviors. Judgment call — if the "ands" represent different system actions, split.

Activity 2 — Happy-Path AC

Quads • 50 minutes

Identify the happy path from your Section 5. Each member solo-drafts 3 AC.
Pool, de-dupe, refine to 3–5 final.

- 5 min: identify the happy path
- 15 min: solo drafts (3 each)
- 15 min: pool + refine
- 5 min: 5-failure-mode check



5 + 15 + 15 + 5

Block 3 — Sad & Weird Paths

Where TPMs earn their seat

Sad-Path Generator

For each happy-path AC, ask:

- What if the **input is invalid**?
- What if the **user lacks permission**?
- What if the **user cancels mid-flow**?
- What if the **data is missing or stale**?

```
Given a dispatcher with 0 open tickets  
When they tap "Reconcile all"  
Then a non-modal toast appears with text "No tickets to reconcile"  
and the dispatcher remains on the summary screen
```



Weird-Path Generator

For each happy-path AC, ask:

- What if the **network drops** mid-action?
- What if **two users do the same thing simultaneously** (race)?
- What if the action **takes longer than expected** (timeout)?
- What if the **upstream system is down**?
- What if the user is at the **boundary** (max, min, empty)?

```
Given a dispatcher with 47 open tickets (more than modal can list)
When they tap "Reconcile all"
Then the modal opens with the first 25 tickets pre-selected
and a banner reads "22 more tickets – load more"
and a tap on the banner appends the next 25 below
```

Activity 3 — Sad & Weird AC

Quads • 50 minutes

Each member produces 2 sad + 2 weird AC. Pool, cull to 4–6 (2–4 sad, 2–4 weird). Run the 5-failure check.

- 10 min: solo sad-path drafts
- 10 min: solo weird-path drafts
- 10 min: pool + cull
- 10 min: failure-mode check



10 + 10 + 10 + 10

Block 4 — AC Cross-Review

Apply the calibrated eye to another quad's AC



Why Cross-Review Now

Friday's review covers the whole Product Requirements Document (PRD). Today's review covers **just AC**. Two reasons:

1. AC are where most PRD weakness shows up — fixing them mid-week leaves time for the rest
2. Reviewing another's AC sharpens your eye for your own remaining gaps

Today's review is informal: no scoring, no rubric. The point is feedback, not grading.



The Review Protocol

1. **Pair quads** (instructor assigns)
2. **Swap PRDs** (reviewer reads Sections 1–5 first for context, then Section 6)
3. **Identify failures and gaps:** failure mode? path missing? implicit AC unwritten?
4. **Author quad responds:** adopt / defer / push back, with reasoning
5. **Authors revise** + write the resolution note

Reviewer rule: "this could be more specific" is unhelpful. If you say it, you also write the specific version.



The Review-Resolution Note

At the bottom of Section 6, document what changed and why:

```
## Review-resolution note (Day 2)

**Reviewers:** Quad B
**Findings:**
1. AC #3 was vague ("loads fast") – REWROTE with explicit 1-sec target
2. Missing weird-path AC for offline submit – ADDED as AC #11
3. AC #7 was implementation-prescriptive – REWROTE in user terms

**Deferred:**
- Reviewer suggested AC for "user clicks back button" – DEFERRED to NFR
discussion Day 3 (more about state preservation than acceptance)

**Pushed back:**
- Reviewer suggested splitting AC #2 – DECLINED. Original "and"
describes one observable state, not multiple behaviors.
```

Activity 4 — AC Cross-Review

Quad pairs • 55 minutes • Wrap

Pair with another quad. Swap PRDs. Review their AC. Author quad responds. Revise and write the resolution note.

- 5 min: read their Sections 1–5
- 15 min: review their Section 6 with the 5 failures + path coverage in mind
- 10 min: author quad responds
- 15 min: revise + write resolution note



5 + 15 + 10 + 15 · 15-min wrap



Day 2 Wrap

What we built

- 5-failure-mode AC detector
- Happy/sad/weird path coverage
- 8–12 AC per PRD
- Cross-review + resolution note

What opens tomorrow

- **Non-Functional Requirements (NFR)**
- Performance, security, accessibility, observability, compliance
- NFRs as constraints, not wishlists

Bring to Day 3: Your PRD with Sections 1–6 complete. NFRs come tomorrow.

Questions & Reflection

Which one of your AC would survive a hostile engineer reading it?